

Assignment 1 part a

Task 1: Linear Computation Node Implementation

The Linear node was implemented to perform the operation $y = \mathbf{Ax} + \mathbf{b}$, where $\mathbf{A}$ is a weight matrix, $\mathbf{b}$ is a bias vector, and $\mathbf{x}$ is the input. The implementation includes both forward and backward passes with proper gradient computation using matrixes.

Key Implementation Details:

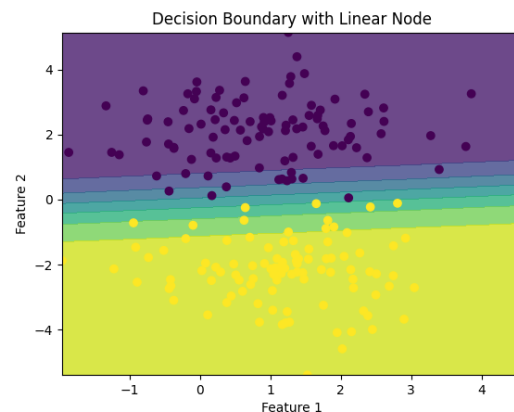
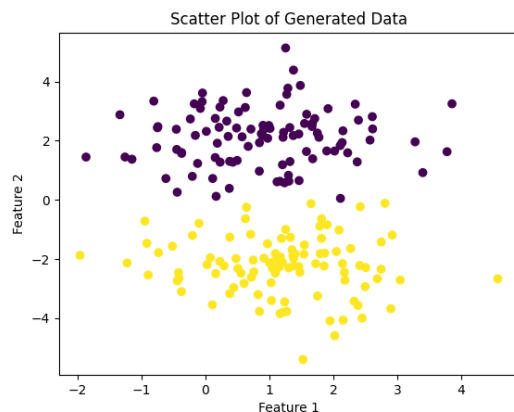
- Forward pass computes matrix multiplication followed by bias addition using `np.dot(A.value, x.value) + b.value`
- Backward pass correctly implements chain rule gradients for all three inputs
- Gradient with respect to input: $\partial y / \partial \mathbf{x}$
- Gradient with respect to weights: $\partial y / \partial \mathbf{A}$
- Gradient with respect to bias: $\partial y / \partial \mathbf{b}$

Task 2: Integration with Logistic Regression

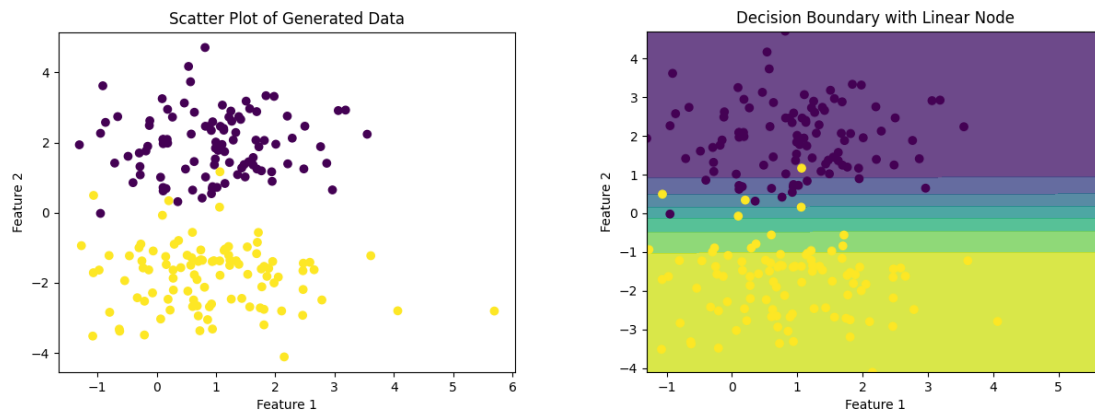
The implemented Linear node replaced the separate Multiply and Addition nodes from the original implementation, this simplification made the computation graph more compact and efficient by condensing multiple operations into a single node.

We ensured that the code still functions correctly after this modification.

Test1:



Test2:



Task 3-4: Batching Implementation and Analysis

The batching implementation transforms input data by transposing feature vectors to shape (n_features, batch_size) . Seven different batch sizes were tested: 1, 2, 4, 8, 16, 32, and 64.

Key Findings from Batch Size Analysis:

1. **Smaller batch sizes** (1-4) provide noisier gradient estimates but can escape local minima more easily
2. **Larger batch sizes** (8-64) produce smoother loss curves.

Critical Fix in Evaluation Code

A bug was identified and fixed in the **evaluation section**.

Original Code:

```
if np.argmax(sigmoid.value) == y_test[i]:  
    correct_predictions += 1
```

Problem: The original code used np.argmax() which is inappropriate for binary classification with a **single sigmoid output**.

Corrected Code:

```
python  
if sigmoid.value[0, 0] >= 0.5:  
    prediction = 1
```

else:

prediction = 0

if prediction == y_test[i]:

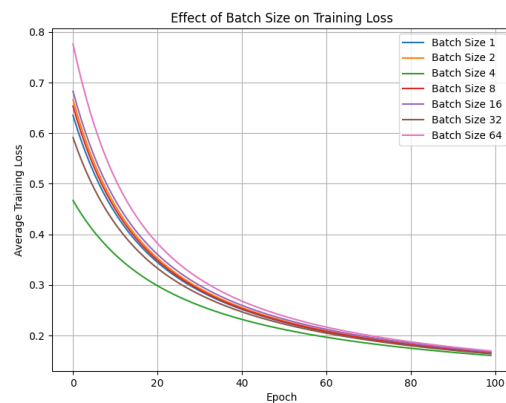
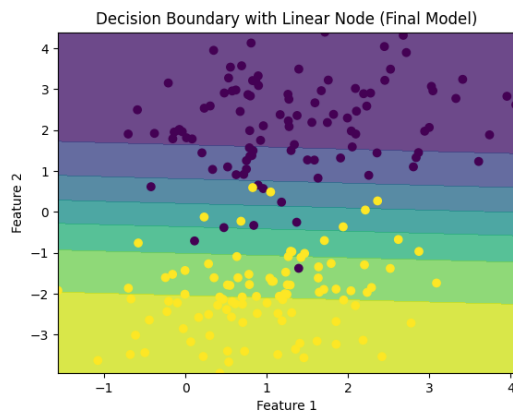
correct_predictions += 1

The fixed code properly implements the **threshold-based decision rule** for binary classification by comparing the sigmoid output against 0.5.

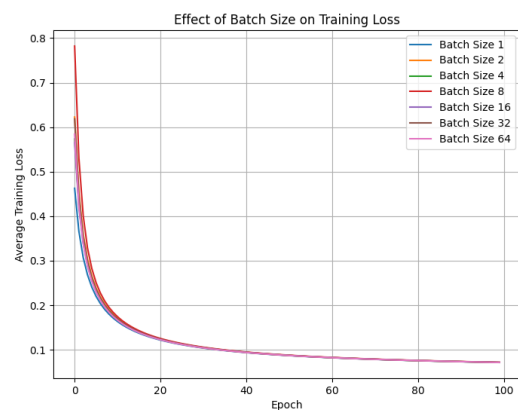
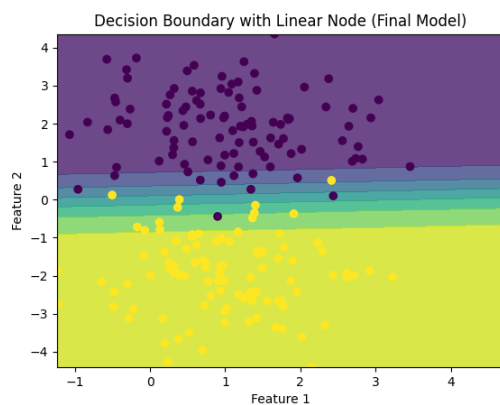
Learning Rate Sensitivity:

Several learning rates was tested.

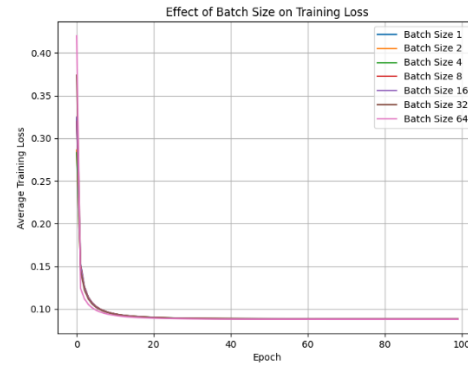
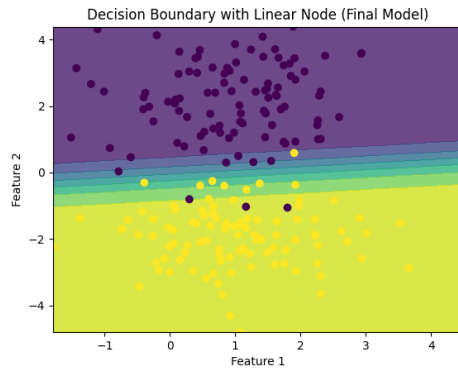
1. Learning rate = 0.0002



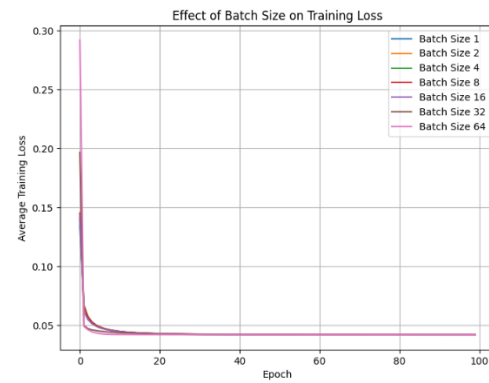
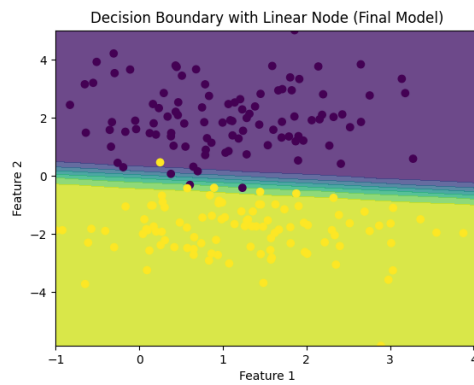
2. Learning rate = 0.002



3. Learning rate = 0.02:



4. Learning rate = 0.01:



Performance Comparison Across Learning Rates

Learning Rate	Convergence Epochs	Final Loss	Batch Size Effect	Training Stability
0.0002	~100	~0.16-0.18	Significant (batch 4 best)	Stable
0.002	~20-25	~0.07-0.08	Minimal (all similar)	Stable
0.02	~10-15	~0.09	Minimal (all similar)	Stable
0.1	~5-10	~0.04	Minimal (all similar)	Stable

Decision Boundary Characteristics

Learning Rate	Separation Quality	Misclassifications	Confidence
0.0002	Moderate	Several near boundary	Moderate
0.002	Good	Few near boundary	Good
0.02	Good	Few near boundary	Good
0.1	Excellent	Minimal	Highest